

Basic Information

Experiment No.: 07

Date: 24-09-2026

Title: Java TreeSet and Its Methods

Objective / Aim

To implement "TreeSet" in Java and demonstrate sorted-set operations such as adding, removing, searching, retrieving boundary elements, and navigation.

System Requirements

Hardware

- Computer/Laptop
- Minimum 4 GB RAM

Software

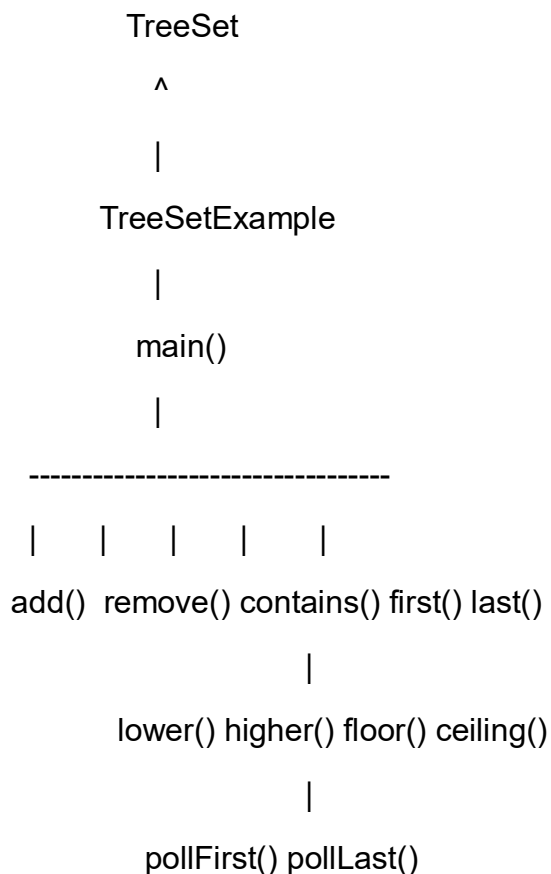
- Java JDK
 - Any Java IDE or Command Prompt
 - Operating System: Windows/Linux/macOS
-

Algorithm / Logic Description

1. Create a "TreeSet" of integers.
2. Add elements using "add()".
3. Display the elements in sorted order.
4. Remove an element using "remove()".
5. Check an element using "contains()".

6. Find the first and last elements using "first()" and "last()".
 7. Find nearby elements using "lower()", "higher()", "floor()", and "ceiling()".
 8. Remove the first and last elements using "pollFirst()" and "pollLast()".
 9. Display the final set.
 10. End the program.
-

UML Class Diagram



Source Code

```
package collectionframework;
```

```
import java.util.TreeSet;

public class TreeSetExample {

    public static void main(String[] args) {

        TreeSet<Integer> set = new TreeSet<>();

        // add()
        set.add(30);
        set.add(10);
        set.add(50);
        set.add(20);
        set.add(40);

        System.out.println("TreeSet: " + set);

        // remove()
        set.remove(20);
        System.out.println("After remove: " + set);

        // contains()
        System.out.println("Contains 30: " +
            set.contains(30));

        // first()
        System.out.println("First: " + set.first());

        // last()
```

```
System.out.println("Last: " + set.last());

// lower()
System.out.println("Lower than 30: " +
    set.lower(30));

// higher()
System.out.println("Higher than 30: " +
    set.higher(30));

// floor()
System.out.println("Floor of 30: " +
    set.floor(30));

// ceiling()
System.out.println("Ceiling of 30: " +
    set.ceiling(30));

// pollFirst()
System.out.println("Removed First: " +
    set.pollFirst());

// pollLast()
System.out.println("Removed Last: " +
    set.pollLast());

System.out.println("Final TreeSet: " + set);
}
}
```

Compilation & Execution Commands

Save the file as:

TreeSetExample.java

Since the program contains:

package collectionframework;

the file should be inside the "collectionframework" folder.

Compile:

javac TreeSetExample.java

Run:

java collectionframework.TreeSetExample

Output Screen

TreeSet: [10, 20, 30, 40, 50]

After remove: [10, 30, 40, 50]

Contains 30: true

First: 10

Last: 50

Lower than 30: 10

Higher than 30: 40

Floor of 30: 30

Ceiling of 30: 30

Removed First: 10

Removed Last: 50

Final TreeSet: [30, 40]

Result

The "TreeSet" class was successfully implemented, and operations such as "add()", "remove()", "contains()", "first()", "last()", "lower()", "higher()", "floor()", "ceiling()", "pollFirst()", and "pollLast()" were successfully demonstrated.